



**UNIVERSIDAD
DON BOSCO**

Facultad de Ingeniería

Escuela de Ingeniería en Computación

Asignatura: Desarrollo de Aplicaciones con Web Frameworks -
DWF 404

Título del proyecto: Infinity Sky

Ciclo / Grupo: 03 – 2026 / G05L

Facilitador: Ing. Juan Carlos Menjivar Ramírez

Estudiantes:

Tiffany Nohemi Benitez Reyes - BR250073

Diego Alejandro Cruz Campos - CC251293

Nancy Yamileth Flores Recinos - FR230188

Daniela Guadalupe Hernandez Mejia - HM250077

Javier Alexander Ramos Garcia – RG251044

Manual Técnico – Airline System API REST

1. Introducción

El presente documento describe la estructura técnica, funcionamiento y componentes principales del proyecto **Airline System API REST**, desarrollado con Spring Boot y orientado a la administración de aerolíneas, vuelos, pasajeros, reservaciones y pagos.

La aplicación implementa una arquitectura basada en API REST, permitiendo la comunicación mediante peticiones HTTP utilizando formato JSON.

2. Objetivos

Objetivo General

Desarrollar una API REST segura y escalable para la gestión de un sistema de aerolíneas.

Objetivos Específicos

- Gestionar aerolíneas.
- Gestionar vuelos.
- Administrar pasajeros.
- Controlar reservaciones.
- Gestionar pagos.
- Implementar autenticación y autorización con JWT.
- Proteger rutas mediante Spring Security.
- Documentar la API mediante Swagger/OpenAPI.

3. Descripción General del Sistema

El sistema Airline System API REST es una aplicación backend desarrollada para automatizar y administrar los procesos principales de una aerolínea.

La API permite centralizar la información relacionada con vuelos, pasajeros, reservaciones, pagos y autenticación de usuarios mediante una arquitectura REST moderna.

El proyecto fue desarrollado utilizando Spring Boot como framework principal, implementando seguridad JWT y persistencia de datos con Spring Data JPA y MySQL.

La aplicación trabaja bajo el modelo cliente-servidor, permitiendo que aplicaciones frontend o clientes externos consuman los servicios mediante endpoints HTTP.

Características Principales

- Arquitectura RESTful.
- Seguridad con JWT.
- Persistencia con JPA/Hibernate.
- Validación de datos.
- Manejo centralizado de excepciones.
- Documentación automática con Swagger.
- Arquitectura escalable en capas.

4. Arquitectura del Proyecto

La aplicación está diseñada bajo una arquitectura multicapa:

- **Controladores (Controllers):** Gestionan las peticiones HTTP.
- **Servicios (Services):** Contienen la lógica de negocio.
- **Repositorios (Repositories):** Acceso a la base de datos.
- **Entidades (Entities):** Representan las tablas de la base de datos.
- **DTOs:** Objetos para transferencia de datos.
- **Configuraciones (Config):** Seguridad, JWT y Swagger.

La API permite:

- Registro e inicio de sesión.
- Administración de vuelos.
- Administración de pasajeros.
- Creación y cancelación de reservaciones.
- Gestión de pagos.
- Administración de aerolíneas.

La arquitectura implementada sigue el patrón multicapa:

Cliente → Controller → Service → Repository → Base de Datos

Descripción de Capas

Controller

Recibe las solicitudes HTTP y devuelve respuestas JSON.

Service

Implementa toda la lógica de negocio del sistema.

Repository

Gestiona el acceso y manipulación de datos utilizando Spring Data JPA.

Entity

Representa las tablas de la base de datos.

DTO

Permite transportar datos entre cliente y servidor.

Security

Gestiona autenticación y autorización mediante JWT.

5. Módulos Implementados en la API REST

4.1 Módulo de Autenticación

Controlador:

- AuthController

Funciones:

- Registro de usuarios.

- Inicio de sesión.
- Generación de token JWT.
- Protección de endpoints.

Endpoints principales:

Método	Endpoint	Descripción
POST	/api/auth/register	Registrar usuario
POST	/api/auth/login	Iniciar sesión

4.2 Módulo de Aerolíneas

Controlador:

- `AirlineController`

Funciones:

- Crear aerolínea.
- Consultar aerolíneas.
- Actualizar información.
- Eliminar registros.

Endpoints principales:

Método	Endpoint
GET	/api/airlines
GET	/api/airlines/{id}
POST	/api/airlines
PUT	/api/airlines/{id}
DELETE	/api/airlines/{id}

4.3 Módulo de Vuelos

Controlador:

- `FlightController`

Funciones:

- Registrar vuelos.

- Consultar vuelos.
- Actualizar vuelos.
- Eliminar vuelos.

Endpoints principales:

Método	Endpoint
GET	/api/flights
GET	/api/flights/{id}
POST	/api/flights
PUT	/api/flights/{id}
DELETE	/api/flights/{id}

4.4 Módulo de Pasajeros

Controlador:

- PassengerController

Funciones:

- Registrar pasajeros.
- Consultar pasajeros.
- Editar información.
- Eliminar pasajeros.

Endpoints principales:

Método	Endpoint
GET	/api/passengers
GET	/api/passengers/{id}
POST	/api/passengers
PUT	/api/passengers/{id}
DELETE	/api/passengers/{id}

4.5 Módulo de Reservaciones

Controlador:

- ReservationController

Funciones:

- Crear reservaciones.
- Consultar reservaciones.
- Cancelar reservaciones.

Endpoints principales:

Método	Endpoint
GET	/api/reservations
POST	/api/reservations
PUT	/api/reservations/cancel/{id}

4.6 Módulo de Pagos

Controlador:

- PaymentController

Funciones:

- Registrar pagos.
- Gestionar transacciones.
- Asociar pagos con reservaciones.

5. Herramientas Utilizadas

IDE Utilizado

- IntelliJ IDEA

Lenguaje de Programación

- Java 17

Framework Principal

- Spring Boot 3.5.14

Gestor de Dependencias

- Maven

Base de Datos

- MySQL

Gestor de Base de Datos

- MySQL Workbench

Herramientas Complementarias

- Swagger OpenAPI
- Spring Security
- JWT
- Lombok
- GitHub
- Postman

6. Dependencias y Versiones

Dependencias principales encontradas en el archivo `pom.xml`:

Dependencia	Versión
Spring Boot Starter Web	3.5.14
Spring Boot Starter Data JPA	3.5.14
Spring Boot Starter Security	3.5.14
Spring Boot Starter Validation	3.5.14
MySQL Connector J	Runtime
Lombok	Última compatible
JWT (jjwt-api)	0.11.5
JWT (jjwt-impl)	0.11.5
JWT (jjwt-jackson)	0.11.5
SpringDoc OpenAPI	2.8.5

7. Configuración de Base de Datos

Configuración encontrada en `application.properties`:


```
spring.datasource.url=jdbc:mysql://localhost:3306/airline_system
spring.datasource.username=root
spring.datasource.password=
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Base de datos utilizada:

- airline_system

Puerto del servidor:

```
server.port=8080
```

8. Diagrama Relacional de la Base de Datos

Entidades Principales

- User
- Airline
- Aircraft
- Flight
- Passenger
- Reservation
- Payment

Relación General

USER
└─ ROLE

AIRLINE
└─ AIRCRAFT
└─ FLIGHT
└─ RESERVATION
└─ PASSENGER
└─ PAYMENT

Descripción de Relaciones

Entidad	Relación
Airline	Tiene múltiples Aircraft
Aircraft	Tiene múltiples Flight
Flight	Tiene múltiples Reservation
Passenger	Puede tener múltiples Reservation
Reservation	Puede tener un Payment
User	Posee un Role

9. Estructura del Proyecto

Paquetes Principales

```
com.infinitsky
├── config
├── controller
├── dto
│   ├── request
│   └── response
├── entity
├── repository
├── security
├── service
│   └── impl
├── exception
└── util
```

Descripción de Paquetes

Paquete	Función
config	Configuraciones generales del sistema
controller	Endpoints REST
dto	Objetos de transferencia
entity	Entidades JPA
repository	Acceso a datos
security	Configuración JWT y seguridad

service	Lógica de negocio
service.impl	Implementaciones de servicios
exception	Manejo de excepciones
util	Utilidades auxiliares

10. Seguridad Implementada

La aplicación implementa autenticación y autorización utilizando:

- Spring Security
- JWT (JSON Web Token)

Características:

- Protección de rutas.
- Autenticación mediante token.
- Roles de usuario:
 - ADMIN
 - USER

11. Usuario Predefinido para Pruebas

La aplicación crea automáticamente un usuario administrador mediante la clase:

- AdminInitializer

Credenciales

Rol	Correo	Contraseña
ADMIN	admin@gmail.com	123456

12. Documentación Swagger

La API incluye documentación automática con Swagger/OpenAPI.

URL de acceso:

<http://localhost:8080/swagger-ui/index.html>

13. Repositorio del Proyecto

URL del repositorio

Agregar aquí la URL oficial del repositorio GitHub:

https://github.com/USUARIO_GITHUB/airline-system

14. Ejecución del Proyecto

Requisitos Previos

- Java 17
- Maven
- MySQL Server
- IDE compatible

Pasos para Ejecutar

1. Clonar el repositorio.
2. Crear la base de datos:

```
CREATE DATABASE airline_system;
```